

リファクタリングのためのトークン節約術

# ボーイスカウトルールで メモリやスキルを改善しよう

エージェントの迷いを減らしてトークン節約

azukiazusa ・ 2026.07.16

# 自己紹介

---

- ▶ **azukiazusa**
- ▶ Frontend Engineer
- ▶ <https://azukiazusa.dev>
- ▶ 週に 1 回、Web 開発と AI の記事を書いています
- ▶ FE（フロントエンド | ファイアーエムブレム）が好き



# エージェントの迷いは、トークンの浪費

---

- ▶ タスクの遂行がスムーズに進められないと、余計なトークンを消費する
  - ▶ 探索・試行錯誤・失敗からのリトライ
- ▶ 同じパターンのコードを実装するのに、毎回同じ調査をさせている
- ▶ 間違ったコードへの手直し指示

「迷い」を減らすことが、そのままトークン節約になる

# 迷いの多くの原因はプロジェクトの暗黙知や典型作業の手順不足

---

- ▶ プロジェクト特有のルールや暗黙知が CLAUDE.md / AGENTS.md に書かれていない
- ▶ 典型的に繰り返す作業の手順がスキルとして明文化されていない

わかっているが、なかなか言語化されなかったり、整理の時間が取れない

# ボーイスカウトルール 来たときよりも美しく

自分が通った場所を、来たときよりも少しでも良くして帰る

Robert C. Martin 「ボーイスカウト・ルール」『プログラマが知るべき97のこと』

- ▶ 機能追加やバグ修正をした際、ついでに少しでもリファクタリングする
- ▶ エージェント時代は **CLAUDE.md / AGENTS.md** やスキルも同じルールの対象になる
- ▶ タスクのついでに、メモリやスキルを少しでも良くして帰る

# 手を入れるポイントはエージェントが迷った箇所

---

- ▶ タスクの様子を観察して、**迷っていた箇所・失敗した箇所**を見つける
- ▶ ハーネスが未整備の初期ほど**細かく観察**し、整備が進んだら**任せていく**
- ▶ マネジメントと同じ
  - ▶ 新人が入ったときはマイクロマネジメントで細かく見て、徐々に任せてマクロマネジメントに移行する

# 事例 1: 暗黙知を CLAUDE.md に書いたら一発で正しいコードに

課題: トラッキング用のデータ属性はプロジェクト特有のルール。何も伝えないと付けてくれず、**毎回後から手直しを指示**していた

## ## トラッキング属性のルール

ユーザー操作を計測するため、ボタンやリンクなどのインタラクティブ要素には必ず以下の属性を付与する

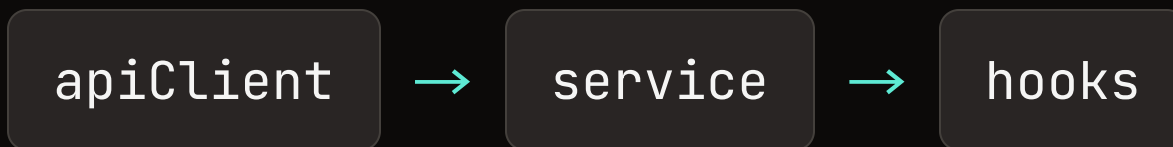
```
- `data-tracking-id="<画面名>-<操作名>"`
```

結果: CLAUDE.md にルールを明文化 → 最初から属性付きのコードが生成される

## 事例 2: API 追加のたびに、同じ調査を繰り返していた

---

- ▶ プロジェクトのフロントエンドから API を呼び出すとき、apiClient → service → hooks というレイヤー構造になっている



- ▶ API 追加のたびに、**このレイヤー構造の調査から**作業が始まっていた
- ▶ 各レイヤーで**テストを書くルール**が守られないことがあった
- ▶ service 追加時に必要な作業が漏れて**失敗することが何度か**あった



# 手順をスキル化する

```
---  
name: add-api  
description: フロントエンドに API 呼び出しを追加する手順  
---
```

1. apiClient にエンドポイントの型と定義を追加
2. service に変換ロジックを実装し、テストを書く
3. hooks から service を呼び出し、テストを書く
4. `npm run typecheck && npm run test` で検証する

- ▶ 事前調査なしで**すぐに作業へ着手**できるようになった
- ▶ 手順と検証方法が明文化され、**作業漏れによる失敗が減った**

# スキルはエージェント自身に書かせる

今行った作業と私からのフィードバックを元に、  
次回から実行できるスキルとして保存して

- ▶ タスク完了直後ならコンテキストが残っている
- ▶ タスクの帰り際にスキルを 1 つ残す = ボーイスカウトルールを片手間にできる

## 改善はタスクと同じコミット / PR に含める

---

- ▶ ついでのリファクタリングと同じようにメモリ・スキルの差分もそのまま PR へ
- ▶ 「改善のための専用タスク」を作ると優先度が下がり、結局やらなくなる
- ▶ 日常の開発の中で少しずつコード品質を向上させられる

# 削るのもボーイスカウトルール

---

- ▶ メモリやスキルは読み込まれるだけでトークンを消費する
- ▶ 使われないものは定期的に掃除することが大事
  - ▶ 言語規約や一般論は書かず、プロジェクト特有の暗黙知を中心に書く
  - ▶ 汎用スキルは個人インストールと重複しやすい — 例: PR 作成スキル

# まとめ

---

- ▶ エージェントの「迷い」はトークンの浪費 — **迷った箇所がメモリ・スキルの改善ポイント**
- ▶ 暗黙知は CLAUDE.md へ、繰り返し作業はスキルへ — ボーイスカウトルールで少しずつ改善
- ▶ ただ追加するだけでなく、一般論や重複は削るなど継続的な改善を

THANK YOU

ご清聴ありがとうございました

azukiazusa.dev